Midterm Report: Comparative Study of Post-Quantum Key-Exchange Mechanisms

Hamdan Al-Jabri (Neptun QUTC51)
Cybersecurity Lab 1, ELTE Faculty of Informatics
November 2026

1. Introduction

Data encrypted today can be recorded and decrypted later, once a large quantum computer exists. NIST has therefore standardised ML-KEM (FIPS 203, 2024) and selected HQC as a backup (2025), and browsers already use the hybrid X25519MLKEM768. The available post-quantum KEMs, however, differ by orders of magnitude in speed and size, and their implementations differ in quality.

The project compares them in four experiments: **E1** performance, **E2** real TLS 1.3 handshakes, **E3** timing leaks, and **E4** correctness. This report covers E4 and E1. Code and data are public at `https://github.com/hsh1994-ux/pq-kem-study`.

2. Background

2.1 Key-encapsulation mechanisms

A KEM lets two parties agree on a shared secret in three steps:

1. **KeyGen**: the client makes a key pair and sends the public key.
2. **Encaps**: the server makes a random secret, locks it with the public key and sends it back (the ciphertext).
3. **Decaps**: the client unlocks it with its private key. Both sides now hold the same secret.

2.2 The four methods: structured and unstructured

Post-quantum KEMs rest on two main kinds of hard maths problems:

- **Lattices:** the public key hides a secret behind a large system of linear equations with small random errors added (the *Learning With Errors* problem). Without the secret, removing the errors is believed to be hard, even for a quantum computer.
- **Error-correcting codes:** the public key is a disguised version of a code that the owner can decode efficiently. The ciphertext is a message with deliberate errors added; without the secret disguise, correcting those errors is believed to be hard. This idea goes back to McEliece in 1978.

Both problems need a large matrix as the public key. The difference between the methods is **how that matrix is built**:

- **Structured:** the matrix follows a repeating pattern, so it can be described by a few short rows instead of being stored in full. In ML-KEM the matrix entries are polynomials, and one polynomial stands for a whole block of the matrix; in HQC every row is a cyclic shift of the first row. This makes keys **much smaller** and the arithmetic **much faster**. The price is that the pattern is extra information an attacker could try to use. No practical attack that does so is known today, but the security rests on a slightly stronger assumption.
- **Unstructured:** the matrix is random, with no pattern, and is stored or generated in full. FrodoKEM uses plain Learning With Errors on a large random matrix; Classic McEliece uses binary Goppa codes hidden behind a random-looking matrix, a design that has resisted attacks for over 45 years. Keys are **much larger** and the work is **slower**, but the security depends only on the basic hard problem, with no pattern to exploit. This is why they are seen as the *cautious* choice.

Choosing one method of each kind lets the study answer two questions at once: how lattices compare with codes, and **what the extra caution of unstructured designs costs**. Our own measurements (Section 4) show the size of this cost: at level 3, ML-KEM sends 2,272 bytes and FrodoKEM 31,424 bytes (14× more); HQC sends 13,492 bytes and Classic McEliece 524,316 bytes (39× more, almost all of it the public key).

Method	Based on	Design	Status
ML-KEM	lattices	structured	NIST standard, FIPS 203 (2024) [1]
HQC	codes	structured	selected by NIST as backup (2025) [2]
FrodoKEM	lattices	unstructured	ISO/IEC 18033-2 Amd 2 (2026); recommended by BSI [3,5]
Classic McEliece	codes	unstructured	ISO/IEC 18033-2 Amd 2 (2026); recommended by BSI [4,5]

Table 1. The four KEMs, one of each kind. Each is tested at NIST security levels 1, 3 and 5 (as strong as AES-128, -192 and -256), so 12 versions in total.

2.3 Baselines

- **X25519 DH** [6]: plain Diffie–Hellman, as used in TLS today. It is not a KEM, but it has the same three steps: the client makes a key pair (KeyGen); the server makes its own key pair and computes the secret (Encaps); the client computes the same secret (Decaps). This is the main baseline.
- **X25519 KEM** (DHKEM, RFC 9180 [7]): the same calculation inside the KEM interface, plus one step that hashes the result with both public keys. Measured for comparison.
- **Hybrid X25519MLKEM768** [8]: used by browsers today. It is packaged as one KEM, but inside it is plain X25519 DH plus ML-KEM-768; the two secrets are joined and TLS hashes them later. It stays safe if either part is broken.

3. Method

3.1 Setup

Two computers were used: an **Apple M4** MacBook Air (ARM64, Linux in Docker) and an **Intel i7-1185G7** laptop (x86-64, Linux in WSL2). Both run the same software: liboqs 0.16.0 [9] for the post-quantum KEMs and OpenSSL 3.5.8 for X25519, the hybrid and a second ML-KEM implementation. Only the processor differs.

liboqs was built in two versions: **portable C**, which runs on any processor, and **optimised** code, which uses vector instructions (AVX2 on Intel, NEON on the M4). Optimised code does not exist for every method on every processor:

Method	Intel (AVX2)	Apple M4 (NEON)
ML-KEM	yes	yes
HQC	no	no
FrodoKEM	yes	no
Classic McEliece	yes	no

Table 2. Optimised code in liboqs 0.16.0. Where there is none, only the portable code is used.

3.2 E4: correctness

Before measuring speed, five kinds of checks were run on both computers:

1. **Known answers:** the code gives exactly the expected output for fixed inputs (all 12, both versions).

2. **NIST test vectors:** ML-KEM matches NIST's official FIPS 203 test vectors (ML-KEM only, the other methods have no NIST standard yet).
3. **Random exchanges:** with random inputs both sides always get the same secret (all 12, both versions).
4. **Cross-version:** keys made by the portable code work with the optimised code, and the other way round (all 12).
5. **Cross-library:** liboqs and OpenSSL ML-KEM work together (ML-KEM only, OpenSSL does not implement the other three).

3.3 E1: performance

- **Timing:** KeyGen, Encaps and Decaps were each run **1,000 times** and the median is reported. One whole key exchange = KeyGen + Encaps + Decaps.
- **Classic McEliece key generation was run 100 times, not 1,000.** One run takes 40–500 ms, so 1,000 runs would take hours. Its time also varies a lot, because it builds a random key and starts again when that fails (about 7 times in 10); 20 runs gave an unstable median, 100 runs give a stable one.
- **Fair conditions:** the timer was calibrated at the start of every run, the processor was warmed up for 200 ms before timing (the M4 needs about 90 ms to reach full speed), and there was a 30-second pause between methods so the fanless MacBook could cool down. The processor's clock speed was also measured, so results can be given in CPU cycles and compared with published numbers.
- **Also recorded:** bytes sent over the network and peak memory (stack) of each step.
- **Comparisons** with X25519 always use the faster of the two code versions.

4. Results

4.1 E4: correctness

All **69 checks (1,332 test cases) pass on both computers**. The optimised code gives exactly the same results as the portable code, so comparing their speed is fair, and liboqs agrees with NIST's official vectors and with OpenSSL's independent ML-KEM.

4.2 E1: performance

Table 3 gives one whole key exchange for every method. **Note:** the time of each step separately (KeyGen, Encaps, Decaps, both code versions) was also measured and tabulated; it is in the project repository.

Main findings:

- **ML-KEM is faster than today's X25519** at every level: ML-KEM-768 needs 0.45× (M4) and 0.29× (Intel) of the X25519 time. Its cost is size: 2,272 bytes instead of 64 (35×), still small enough for one TLS message.
- **The hybrid costs 1.6–1.7× X25519**, about X25519 plus ML-KEM added together. The KEM form of X25519 is 10–14% slower than plain DH.
- **The cautious methods are much more expensive in both time and size.** At level 1: HQC 54–65× X25519, FrodoKEM 19–43×, Classic McEliece 337–1,047×. McEliece's public key (0.26–1 MB) is too large for a TLS key share, and it needs up to 1.5 MiB of stack, against 10–21 KiB for ML-KEM.

			Apple M4 (NEON code)			Intel i7 (AVX2 code)		
Method	Level	Bytes	Portable	Optimised	vs X25519	Portable	Optimised	vs X25519
X25519 DH (today)	–	64	–	62.5 μs	1.00×	–	115 μs	1.00×
X25519 KEM (DHKEM)	–	64	–	71.5 μs	1.14×	–	126 μs	1.10×
Hybrid X25519MLKEM768	–	2,336	–	100 μs	1.60×	–	191 μs	1.66×
ML-KEM-512	1	1,568	26.8 μs	17.6 μs (1.53×)	0.28×	64.3 μs	20.8 μs (3.1×)	0.18×
ML-KEM-768	3	2,272	42.7 μs	28.0 μs (1.53×)	0.45×	98.6 μs	33.2 μs (3.0×)	0.29×
ML-KEM-1024	5	3,136	64.5 μs	42.4 μs (1.52×)	0.68×	143 μs	46.4 μs (3.1×)	0.40×
HQC-1	1	6,674	3.39 ms	–	54×	7.41 ms	–	65×
HQC-3	3	13,492	10.1 ms	–	162×	22.3 ms	–	195×
HQC-5	5	21,658	22.3 ms	–	357×	52.8 ms	–	461×
Frodo-640	1	19,368	2.70 ms	–	43×	6.33 ms	2.23 ms (2.8×)	19×
Frodo-976	3	31,424	6.10 ms	–	98×	13.9 ms	4.61 ms (3.0×)	40×
Frodo-1344	5	43,216	11.0 ms	–	176×	25.3 ms	7.58 ms (3.3×)	66×
McEliece-348864	1	261,216	65.5 ms	–	1,047×	90.1 ms	38.7 ms (2.3×)	337×
McEliece-460896	3	524,316	159 ms	–	2,536×	276 ms	113 ms (2.5×)	983×
McEliece-6688128	5	1,045,200	461 ms	–	7,375×	592 ms	134 ms (4.4×)	1,166×

Table 3. One whole key exchange (KeyGen + Encaps + Decaps), median. Portable = portable C; ($n\times$) after an optimised time = how many times faster it is than portable C; – = no optimised code. "vs X25519" compares with X25519 DH on the same computer, using the faster version: below 1.00× means faster than X25519.

- **Optimised code matters as much as the method.** On Intel it makes ML-KEM and FrodoKEM about 3× faster and McEliece Decaps 510–1,400× faster. On the M4 only ML-KEM has optimised code (1.5×). As a result the M4, which is about 2.2× faster on portable code, is slower than the older Intel chip for FrodoKEM and McEliece.
- **Agreement with published results:** measured in CPU cycles, HQC and McEliece Encaps/Decaps agree with the designers' numbers within 7–20%; ML-KEM and FrodoKEM are faster than older published figures because liboqs uses newer code.

5. Limitations

Both computers run Linux in a virtual machine (Docker, WSL2), and there is one computer per architecture. Classic McEliece key generation varies strongly from run to run because of its random retries, and McEliece-348864 key generation takes 2.6× the published cycle count, which is still being investigated. liboqs has no optimised HQC code, so HQC's gap to ML-KEM is partly an implementation gap.

6. Remaining work

E2: real TLS 1.3 handshakes with each KEM, also under network delay and packet loss. **E3**: tests for secret-dependent timing (dudect, Valgrind, KyberSlash-style division checks). Then the final report and the demo at the mini-conference.

References

[1] NIST, *FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard*, 2024.

[2] NIST, *IR 8545: Status Report on the Fourth Round of the NIST PQC Standardization Process*, 2025.

[3] E. Alkim et al., *FrodoKEM specification*; ISO/IEC 18033-2:2006/Amd 2:2026.

[4] D. J. Bernstein et al., *Classic McEliece: conservative code-based cryptography*; ISO/IEC 18033-2:2006/Amd 2:2026.

[5] BSI, *TR-02102-1: Cryptographic Mechanisms: Recommendations and Key Lengths.*

[6] A. Langley, M. Hamburg, S. Turner, *RFC 7748: Elliptic Curves for Security*, 2016.

[7] R. Barnes et al., *RFC 9180: Hybrid Public Key Encryption*, 2022.

[8] K. Kwiatkowski et al., *Post-quantum hybrid ECDHE-MLKEM key agreement for TLSv1.3*, IETF draft-ietf-tls-ecdhe-mlkem.

[9] Open Quantum Safe, *liboqs 0.16.0*, `https://openquantumsafe.org`.